

**NANYANG
TECHNOLOGICAL
UNIVERSITY**

SINGAPORE

SCSE21 - 0010

Deep Learning for Computer Chess (Part 1)

Supervisor: A/P He Ying

Submitted in Partial Fulfillment of the Requirements
for the Degree of Bachelor of Engineering in Computer
Science of the Nanyang Technological University

by

Arora Manav

School of Computer Science and Engineering

2022

Abstract

This report encompasses the implementation of two state-of-the-art machine learning algorithms for evaluating chess positions. The first algorithm makes use of artificial neural networks and manual feature representation thus closely following the implementation and architecture of Matthew Lai's Giraffe. Giraffe learns to play chess largely by self-play and derives its own rules based on the data [1]. Giraffe was implemented as a 7 class classification problem on a dataset of over 10,000 grandmaster level games. Four different implementations of Giraffe were explored covering two different architectures and the effects of regularization on the model performance.

The second algorithm implemented goes through an unsupervised learning phase to perform feature extraction followed by a supervised learning phase thus replicating David Eli's DeepChess. DeepChess evaluates chess positions using a deep neural network without any a priori knowledge regarding the rules of chess. DeepChess is implemented as a siamese network of two disjoint deep belief networks connected to each other by fully connected layers [2]. This architecture was implemented as a binary classification problem on the same dataset as Giraffe and also on a larger dataset of LiChess games. Different implementations of DeepChess covering different training methodologies and parameter sets were executed.

Acknowledgement

I would like to express my gratitude to Professor He Ying for his constant guidance and valuable feedback throughout the entirety of this project. His support and feedback have been essential to the development of this project. I would also like to thank Nanyang Technological University for providing me with the resources and the opportunity to conduct this research.

Table of Contents

Abstract	2
Acknowledgement	3
Table of Contents	4
List of Figures	6
List of Tables	8
1. INTRODUCTION	9
1.1. Background	9
1.2. Objective	10
2. LITERATURE REVIEW	11
2.1. Traditional Chess Engines	11
2.1.1. Minimax Search	11
2.1.2. Alpha Beta Search	12
2.1.3. Search Enhancement	12
2.2. Modern Chess Engines	12
3. IMPLEMENTATION	14
3.1. Giraffe	14
3.1.1. Dataset Creation	14
3.1.2. Feature Representation	16
3.1.3. Data Preparation	19
3.1.4. Model Architecture	20
3.2. DeepChess	24

3.2.1.	Dataset Creation	24
3.2.2.	Deep Belief Networks	25
3.2.2.1.	Autoencoder Structure	25
3.2.2.2.	DBN Structure	26
3.2.3.	Model Architecture	27
4.	EXPERIMENTS	29
4.1.	Giraffe	29
4.1.1.	Experiment 1: Segmented Giraffe vs Combined Giraffe	29
4.1.2.	Experiment 2: Effects of Regularization on Segmented and Combined Giraffe	33
4.2.	DeepChess	36
4.2.1.	Experiment 1: Effect of Optimizer and Loss Function in DBN training (pos2vec)	37
4.2.2.	Experiment 2: Effects of Freezing Decoder Layers in DBN training (pos2vec)	41
4.2.3.	Experiment 3: Effects of a Bigger Dataset	44
5.	CONCLUSION	47
	References	49
	Appendix	50

List of Figures

Figure	Page
Figure 1. Data distribution of data with respect to labels	16
Figure 2. Sample board for feature representation demonstration	17
Figure 3. Segmented Giraffe abstract architecture	21
Figure 4. Segmented Giraffe complete model architecture	22
Figure 5. Combined Giraffe complete model architecture	23
Figure 6. Stacked autoencoders implementation	26
Figure 7. DeepChess DBN architecture	27
Figure 8. DeepChess architecture	28
Figure 9. Loss vs Epochs curve for Segmented Giraffe	30
Figure 10. Accuracy vs Epochs curve for Segmented Giraffe	30
Figure 11. Loss vs Epochs curve for Combined Giraffe	31
Figure 12. Accuracy vs Epochs curve for Combined Giraffe	32
Figure 13. Loss vs Epochs curve for Segmented Regularized Giraffe	33
Figure 14. Accuracy vs Epochs curve for Segmented Regularized Giraffe	34
Figure 15. Loss vs Epochs curve for Combined Regularized Giraffe	35

Figure 16.	Accuracy vs Epochs curve for Combined Regularized Giraffe	35
Figure 17.	Loss vs Epochs curve for DeepChess with DBN trained using adam optimizer and binary cross-entropy loss	38
Figure 18.	Accuracy vs Epochs curve for DeepChess with DBN trained using adam optimizer and binary cross-entropy loss	38
Figure 19.	Loss vs Epochs curve for DeepChess with DBN trained using SGD optimizer and MSE loss	39
Figure 20.	Accuracy vs Epochs curve for DeepChess with DBN trained using SGD optimizer and MSE loss	39
Figure 21.	Loss vs Epochs curve for DeepChess with DBN trained without freezing decoder layers	41
Figure 22.	Accuracy vs Epochs curve for DeepChess with DBN trained without freezing decoder layers	42
Figure 23.	Loss vs Epochs curve for DeepChess with DBN trained with freezing decoder layers	42
Figure 24.	Accuracy vs Epochs curve for DeepChess with DBN trained with freezing decoder layers	43
Figure 25.	Loss vs Epochs curve for DeepChess trained on LiChess Dataset	45
Figure 26.	Accuracy vs Epochs curve for DeepChess trained on LiChess Dataset	46

List of Tables

Table	Page
Table 1. Label to evaluation condition mapping	15
Table 2. Description of features used in Giraffe feature representation	17
Table 3. Sample Giraffe feature representation	18
Table 4. Class label to numerical value mapping	19
Table 5. Train and test data shapes for Giraffe	20
Table 6. Configuration of the autoencoders used in the DeepChess DBN	25
Table 7. Segmented and Combined Giraffe performance summary	32
Table 8. Segmented Regularized and Combined Regularized Giraffe performance summary	36
Table 9. DeepChess with DBN trained using different sets of parameters performance summary	40
Table 10. DeepChess with DBN trained with and without freezing decoder layers performance summary	43
Table 11. DeepChess trained on LiChess dataset performance summary	46

1. INTRODUCTION

1.1. Background

Chess is a complex, zero-sum strategy game with approximately 10^{50} unique states. This implies that if the evaluation of a single state takes one nanosecond, then the evaluation of all chess positions would take 3×10^{35} years [3]. This complexity gave rise to the chess problem in modern artificial intelligence. The goal of the chess problem is to assign a score to a chess position that correlates to the side's chance of victory. This evaluation is monotonically increasing with respect to the likelihood of winning.

Chess engines were developed to tackle the chess problem. A chess engine is a software program that plays and analyses chess. Engines have been able to play human-level chess for over two decades now using a set of brute-force techniques based on the idea of fixed depth minimax algorithm first developed by John von Neumann in 1928 [4]. This was then adapted for the problem of chess by Claude E. Shannon in 1950 [5]. Engines today are so advanced that the best humans have no realistic chance of defeating a modern chess engine.

Despite the success of modern chess engines, one key limitation of these engines is their inability to learn due to their brute-force nature. Furthermore, humans are much more computationally efficient than computers. These differences can be highlighted through the different ways in which humans and computers play chess. By making use of learning and intuition a chess grandmaster analyzing 3-5 positions per second, performs on par with a supercomputer analyzing about 200 million positions per second (IBM DeepBlue vs Garry Kasparov, 1997) [6].

Top of the line chess engines are a result of years of manual feature selection, hardcoded rules and tuning. Such a selective and niche approach leads to a lack of generalisability and learning

(intuition) while promoting the brute force nature of these algorithms. Recently, with the advent of deep learning and increased computational ability, there has been a shift from these brute force algorithms towards a machine learning-centric approach.

1.2. Objective

The aim of this project is to study state-of-the-art machine learning algorithms such as Matthew Lai's Giraffe and David Eli's DeepChess for evaluating chess positions. Deep neural networks are used in these algorithms to replace human intuition. Giraffe learns to play chess largely by self-play and derives its own rules based on the data. [1]. Whereas DeepChess evaluates chess positions using a deep neural network without any a priori knowledge regarding the rules of chess [2].

Our approach is to create a huge training database of thousands of chess positions from top grandmasters with no domain-specific features or knowledge to train the network. Subsequently, the machine learning algorithms mentioned above will be implemented to create engines that can learn the rules and intricacies of the game by themselves from the database and can evaluate, and analyze chess more efficiently by comprehending complex strategic ideas that are intuitive to humans but have been difficult for computers to grasp for a long time.

2. LITERATURE REVIEW

Chess can be represented as a game tree. The initial state of the game is represented as a root node in the game tree and all possible positions after one move are the immediate successors of the root. Similarly, for each node, all positions after one move are its successors and so and so forth the game tree is created [3]. Making a chess move on the board is equivalent to going down a branch in the chess game tree. Each layer of the tree represents alternative players, i.e. if layer 1 is a white move then layer 2 will correspond to the black side and layer 3 will be white again and so on and so forth.

2.1. Traditional Chess Engines

Based on the game tree representation of chess, the minimax algorithm was developed and adopted for chess. This algorithm was the core behind almost every traditional chess engine.

2.1.1. Minimax Search

In minimax, the root node is referred to as a max node as the player intends to maximize the score evaluation of the game tree. The next layer of nodes is referred to as the min nodes since this player intends to minimize the score evaluation. Max and min nodes alternate in a game tree until a given depth. During the evaluation of a node, the scores of every preceding node in the min layer are calculated as the minimum of their child's value and similarly, max layer nodes are calculated as the maximum of the child's value. This process is repeated until the root node [3].

2.1.2. Alpha Beta Search

In chess there is no need for the complete exploration of the game tree. Omission of certain nodes does not affect minimax evaluation but allows for better performance. Alpha Beta search basically introduces the concept of deep cut-offs to the minimax algorithm to improve performance [3]. This method implies that two values are maintained for each node. These values represent the lower and upper bound for the evaluation function for min and max layers respectively. Any node lying outside these bounds of their respective subtree is left out and does not need to be explored. This process relies heavily on the optimal ordering of leaf nodes as a proper ordering would lead to more substantial cutoffs.

2.1.3. Search Enhancement

Performance of the above-mentioned search algorithms depends on the number of nodes to be evaluated. Sorting and ordering of nodes is one way to bring about these enhancements. These enhancements need to be faster and more efficient than the expensive search evaluations in order for them to make sense. Search enhancements are also known as heuristics. Some commonly used heuristics to improve the performance of search algorithms are killer moves, history heuristics, transposition tables, refutation tables, null moves, etc.

2.2. Modern Chess Engines

Modern chess engines make use of machine learning to learn and play the game. Initially, the chess evaluation functions made use of hand-selected features. Machine learning was introduced into these functions in order to automatically tune the parameters of the hand-selected features or

to use these selected features as input to machine learning algorithms to determine the evaluation.

The advent of deep learning allowed machine learning based engines to go beyond parameter tuning. Deep learning allows for extremely deep, layered, connected and hierarchical networks which are able to perform feature extraction and learn chess evaluation functions. This advent in deep learning combined with the increased processing and computation power available today, deep networks can now be used to evaluate chess positions, pick specific branches to look at and sort and order chess moves.

3. IMPLEMENTATION

3.1. Giraffe

This section provides a detailed description of the Giraffe implementation ranging from the creation of the data to the feature extraction and the network architecture.

3.1.1. Dataset Creation

A dataset of over 10,000 games played at the highest level by five grandmasters (Alexander Alekhine, Garry Kasparov, Anatoli Karpov, Paul Keres, Victor Korchnoi) was constructed. This was implemented by writing a parser in python to read, clean and aggregate these games. Two key chess based data structures were utilized to execute this parser. The first data structure used is called PGN which stands for portable game notation. PGN is a modular data format that stores the moves of the game along with additional information about the game such as player names, game score, date, location, etc. The second data structure used was FEN (Forsyth - Edwards Notation). FEN is useful because it is able to provide a detailed description of one specific position. In addition to a detailed board position description, FEN also holds information regarding castling possibility, whose move it is and the number of moves. Therefore, the parser uses PGN files that contain thousands of games and extracts each individual game and for each game, the parser outputs each position as a FEN string into a consolidated database.

The next step of dataset creation is to label these individual FEN strings by making use of a popular chess engine known as Stockfish at depth 10 [7]. The evaluation method used was centipawn loss. Centipawn is a method used to measure advantage in a chess position. One centipawn represents 100th of a pawn, a value that has no part in chess but it is a useful metric for position evaluation. This centipawn evaluation was then converted to a standard evaluation

by first negating the score wherein black is the current player and then dividing the scores by one hundred.

Once all the positions in the database are labelled by Stockfish, they can then be classified into the following classes: White Winning, White Decisive Advantage, White Better, Equal, Black Better, Black Decisive Advantage, Black Winning. The conditions applied on the Stockfish evaluation to create these labels are listed in the table below.

Label	Condition on Evaluation
White Winning	$\text{Evaluation} > 5$
White Decisive Advantage	$3 < \text{Evaluation} \leq 5$
White Better	$1 < \text{Evaluation} \leq 3$
Equal	$-1 \leq \text{Evaluation} \leq 1$
Black Better	$-3 < \text{Evaluation} \leq -1$
Black Decisive Advantage	$-5 < \text{Evaluation} \leq -3$
Black Wining	$\text{Evaluation} < -5$

Table 1. Label to evaluation condition mapping

The data distribution of the dataset with respect to the above-mentioned labels shows some interesting but expected traits. For instance, most games in the dataset are of an equal evaluation this is primarily because this data consists of games played at the highest level by some of the most accomplished chess grandmasters and most games at that level either end in a draw or are in an equal position for the majority of a match

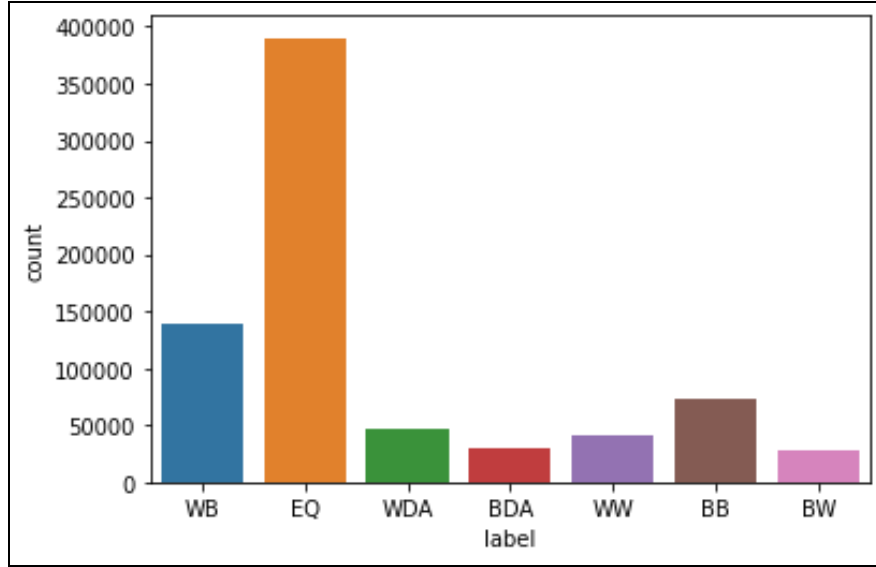


Figure 1. Data distribution of data with respect to labels

3.1.2. Feature Representation

Feature representation is an essential step to prepare the previously obtained labelled dataset for the Giraffe network. The goal is to obtain a low-level representation enough to allow the network to derive its own rules by self-play and be high level enough to make certain useful features easier to learn [1]. Taking this into account, the representation implemented is based on the feature representation from the Giraffe evaluation function. This was implemented using an open-source program written in C# to convert individually labelled FEN strings into a Giraffe like feature representation. The following features are used in the above-mentioned representation:

Feature	Description
Side to Move	Whose move it is, i.e either white or black
Castling Rights	Availability of long and short castling rights for either side
Material Configuration	The number of all pieces of different types
Piece Record	Position and presence of each piece on the board
Sliding Motion	The extent to which each sliding piece can move
Game Maps	The lowest valued attacker and defender corresponding to every square

Table 2. Description of features used in Giraffe feature representation

This representation amounts to a total of 357 features generated to allow the network to derive its own rules by self-play but also make certain useful features like promotions, tempo, piece cohesion, etc easier to learn.

An instance of the feature representation of a sample position is provided below.

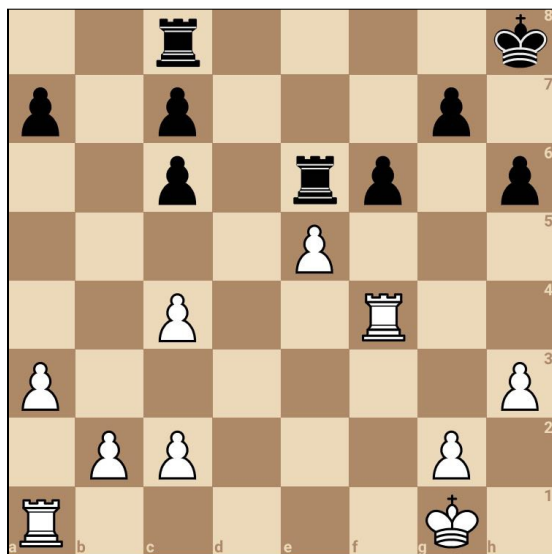


Figure 2. Sample board for feature representation demonstration

Feature	Value
Side to Move	Black
White Long Castle	No
White Short Castle	No
Black Long Castle	No
Black Short Castle	No
White Queens Count	0
White Rooks Count	2
White Bishops Count	0
White Knights Count	0
White Pawns Count	6
Black Queens Count	0
Black Rooks Count	2
Black Bishops Count	0
Black Knights Count	0
Black Pawns Count	6
White Queen 1 Exists	No
White Queen 1 Position	N/A
White Rook 1 Exists	Yes
White Rook 1 Position	a1
White Rook 2 Exists	Yes
White Rook 2 Position	f4
.....	

Table 3. Sample Giraffe feature representation

3.1.3. Data Preparation

Once each FEN string has been converted into a Giraffe-like feature representation, the dataset obtained has 748617 rows each with 357 features. This dataset needs to be prepared in accordance with our network architecture, therefore all the labels mentioned in section 3.1.1 need to be encoded, the numerical features need to be normalized and finally the data need to be split into train and test sets.

The first step of this process is to encode the labels using the Label Encoder provided by the scikit learn library. Each category is encoded to a corresponding numerical value. These mappings are detailed in the table below.

Label	Mapping
Black Better	0
Black Decisive Advantage	1
Black Wining	2
Equal	3
White Better	4
White Decisive Advantage	5
White Better	6

Table 4. Class label to numerical value mapping

The next step is to normalize the numerical data. This is a crucial step as it brings cohesion to the data allowing one to contain the different magnitudes of numerical features. In this implementation, this was achieved by scaling each feature to a range of 0 to 1. This was done by making use of the MinMaxScaler provided by the scikit learn library.

Once the data was encoded and scaled, it was split into training and testing sets using the train test split feature from the sklearn library. The split chosen was 90:10, and the final shapes of the obtained data sets can be seen in the table below.

Dataset	Shape
X_train	(673755, 356)
X_test	(74862, 356)
Y_train	(673755,)
Y_test	(74862,)

Table 5. Train and test data shapes for Giraffe

3.1.4. Model Architecture

This section details the two different neural network architectures that were implemented. The primary implementation also referred to as Segmented Giraffe from here onwards follows the original Giraffe architecture wherein the input data is split into three groups of different modalities and are fed into the network independently. This is done in order to avoid unnecessary connections that bear little to no value and to help prevent overfitting. The three groups are global features, piece centric features and square centric features respectively [1]. This implies that the input layer is not fully connected to the first hidden layer.

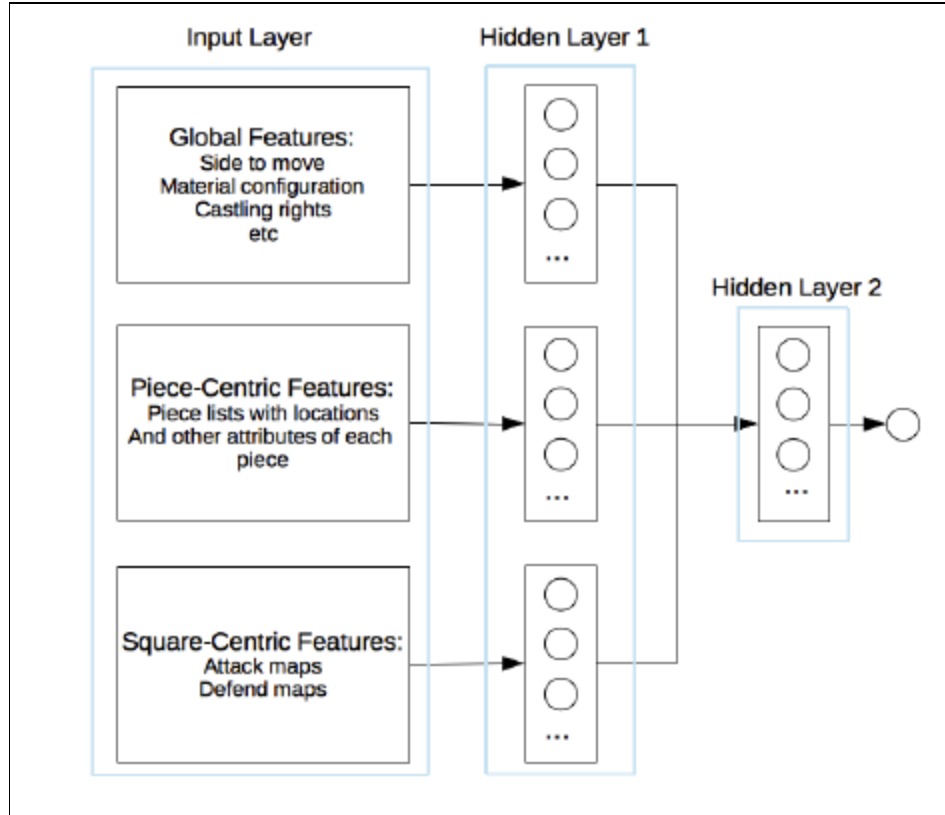


Figure 3. Segmented Giraffe abstract architecture, image courtesy of M. Lai et al. [1]

As mentioned before, in Segmented Giraffe the input layer is split up and fed into the first hidden layer. These components are later concatenated and are then fully connected. This is done to capture interactions between concepts from each individual group. All the hidden layers make use of rectified linear activation.

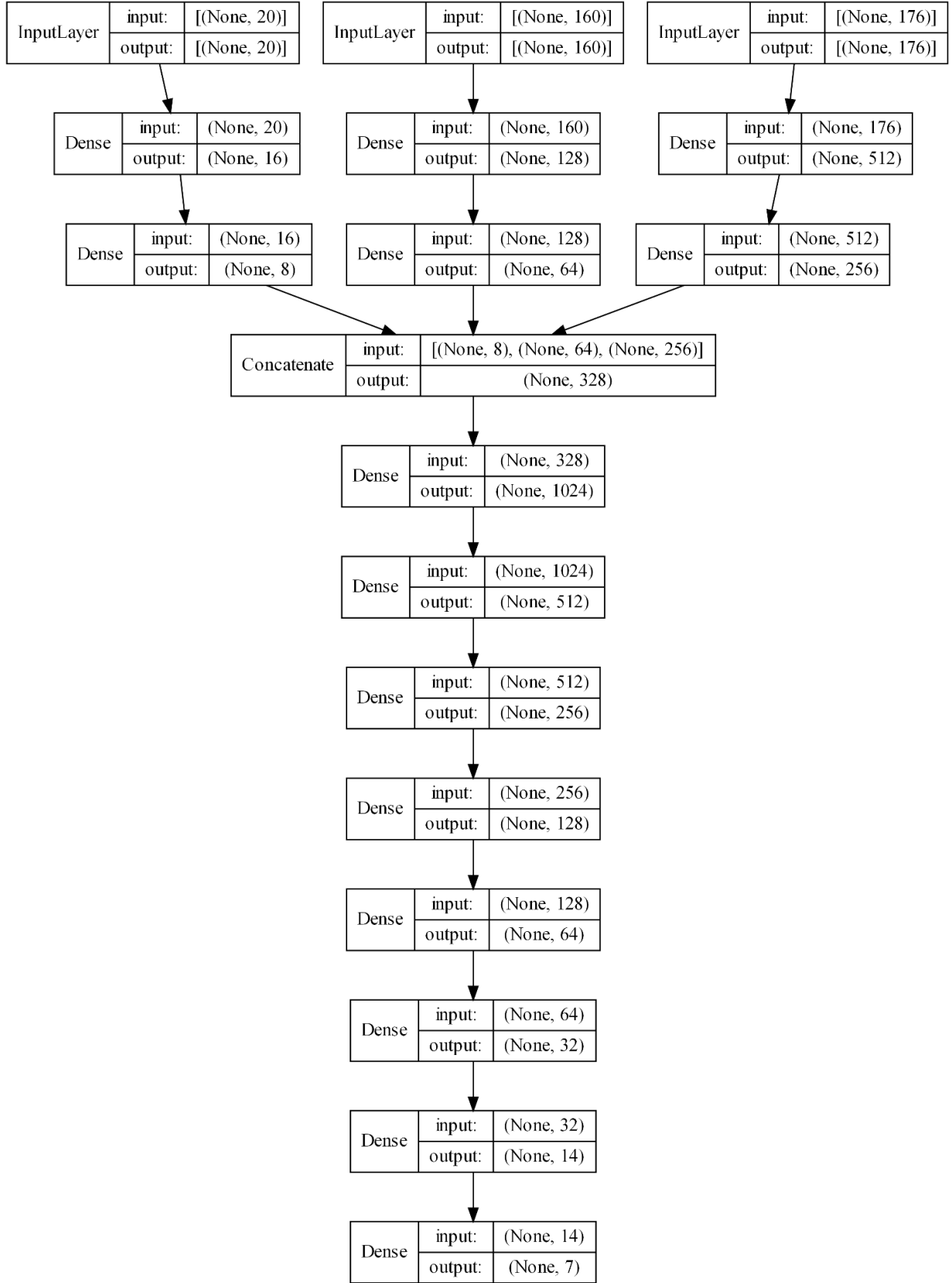


Figure 4. Segmented Giraffe complete model architecture

The second implementation referred to as Combined Giraffe omits the input segregation mentioned in the previous approach. This is primarily implemented to establish a point of comparison and to gauge the effects of splitting the input layer. The architecture used is a simple fully connected layer artificial neural network.

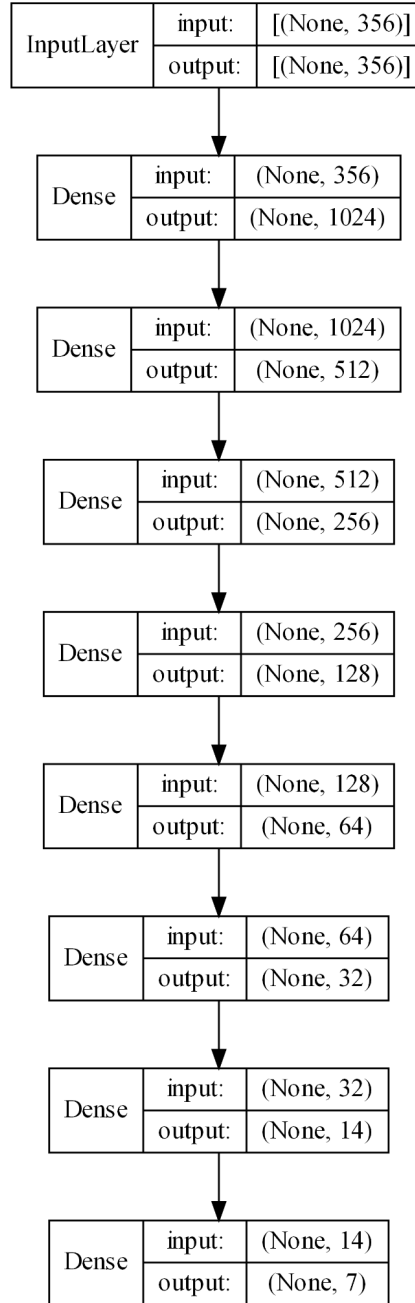


Figure 5. Combined Giraffe complete model architecture

3.2. DeepChess

This section provides a detailed description of the DeepChess implementation ranging from the creation of the data to the network architecture.

3.2.1. Dataset Creation

A dataset of over 10,000 games played at the highest level by five grandmasters (Alexander Alekhine, Garry Kasparov, Anatoli Karpov, Paul Keres, Victor Korchnoi) was constructed. Although the same games were used for both Giraffe and DeepChess, the structure of both these datasets were completely different from each other.

To create the DeepChess dataset, a parser was implemented which extracted 20 positions from each game such that these were neither capture moves nor initial moves from the game. Capture moves were not considered in order to avoid the misleading transient advantage caused by such positions whereas initial moves were not considered because openings in chess are extremely well known and thus don't provide a lot of useful information [2]. Drawn games were omitted for this dataset as it was deemed by the DeepChess authors that they are not beneficial to the model training. The number of moves extracted per game should be 10 according to the DeepChess paper, but since this dataset had relatively less number of games that had a definite winner, this number was increased to 20. This also made sense because games played at a grandmaster level tend to be longer (have more moves played) thus allowing for this increased extraction to make sense.

Once the parser had extracted all the positions, each one was converted to a bitboard representation. Each bitboard is a 773-bit vector that contains all the information regarding the

chessboard for a given position. The bitboard encodes information like side to play, piece types, complete 8x8 board representation and castling rights.

3.2.2. Deep Belief Networks

A Deep Belief Network is a stacked, multilayered network wherein the layers are interconnected.

A deep belief network of stacked autoencoders is the core concept behind the DeepChess architecture.

3.2.2.1. Autoencoder Structure

An autoencoder is a type of neural network that is used to learn mappings of its input so that it can reconstruct it in an unsupervised environment. This implies that an autoencoder follows a bottleneck architecture wherein the first step is to encode the input into a latent low-dimensional code followed by a reconstruction of the input by making use of this latent code. This encoding is carried out by a sub-network known as the encoder, and the reconstruction is done by the decoder. Usually, the decoder is just a laterally inverted encoder.

The DeepChess DBN makes use of four autoencoders. Their size configurations can be seen in the table below.

	Encoder	Latent Code	Decoder
Autoencoder I	773	600	773
Autoencoder II	600	400	600
Autoencoder III	400	200	400
Autoencoder IV	200	100	200

Table 6. Configuration of the autoencoders used in the DeepChess DBN

3.2.2.2. DBN Structure

As mentioned in the previous section, the DeepChess DBN consists of four stacked autoencoders. These autoencoders are stacked such that the next autoencoder is inserted between the latent code and decoder of the previous one. There are two ways such a stacked network can be trained, either by freezing the weights of the previous layer so as to preserve the information they contain for future training rounds or by not freezing the layers and letting them train together. Both these methods were implemented and their comparison can be observed in section 4.2.

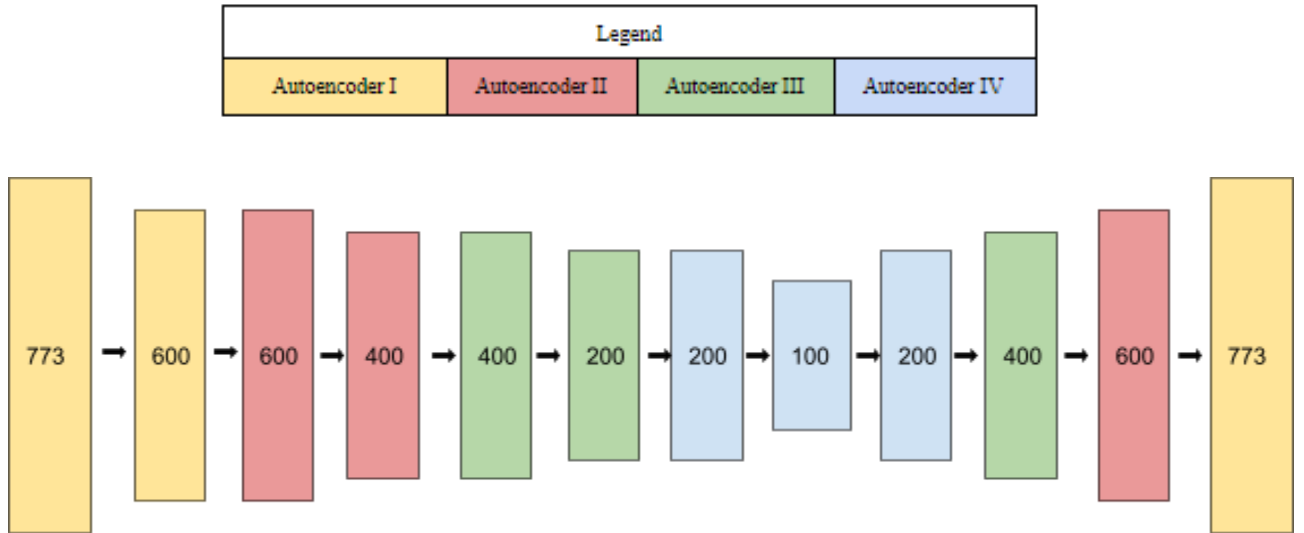


Figure 6. Stacked autoencoders implementation

Once the stacked autoencoders were trained, the code layer from each was extracted and stacked to constitute the DeepChess DBN. The DBN training is carried out layer-wise. This method was termed as pos2vec in the Deep Chess paper and thus the same terminology will be used here [2].

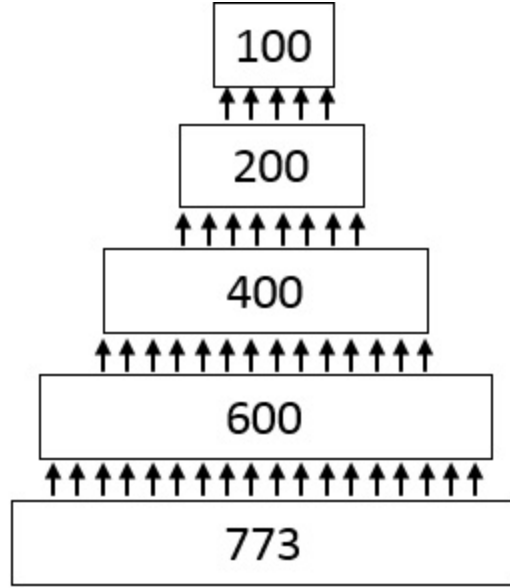


Figure 7. DeepChess DBN architecture, image courtesy of O.E. David et al. [2]

3.2.3. Model Architecture

DeepChess is implemented as a siamese network consisting of two Deep Belief Networks. Both of them are concatenated and passed through four fully connected layers to complete the DeepChess model. The Deep Belief Network used is the one implemented in section 3.2.2.2. The two DBNs are responsible for a higher level feature extraction, these features are then fed into the fully connected layers to evaluate the better position from any given pair of positions [2]. The data used consisted of 110,000 positions, 55,000 of which were white wins and the other half were black wins. This data was split up into training and testing sets. Training data was made up of 100,000 positions with 50,000 wins for each colour. Similarly, the testing data had 10,000 positions with 5,000 wins for each colour. Therefore, the input to the DeepChess model was 50,000 pairs of positions. These are randomly sampled at each epoch to increase the robustness of the model as random sampling increases the amount of potential pairs that can be fed into the network.

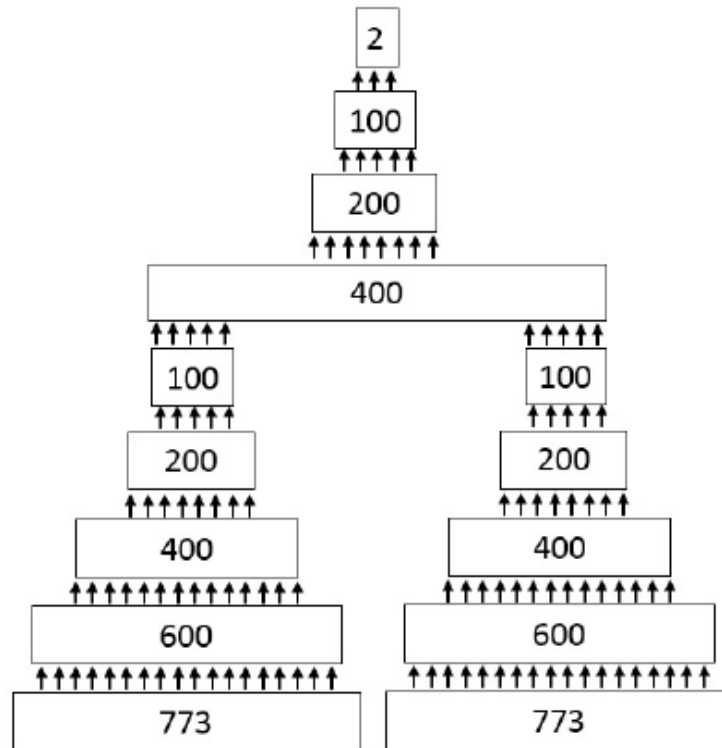


Figure 8. DeepChess architecture, image courtesy of O.E. David et al. [2]

4. EXPERIMENTS

4.1. Giraffe

This section provides a detailed description of the experiments conducted for the Giraffe network architecture.

Giraffe was implemented as a 7 class classification problem with a sparse categorical cross-entropy loss function and adam optimizer. Model checkpoint and early stopping callbacks were also implemented in order to save the best iteration of the model and monitor and prevent overfitting respectively. Overall four different implementations of Giraffe were executed as part of two experiments which are described below.

4.1.1. Experiment 1: Segmented Giraffe vs Combined Giraffe

This experiment was set up in order to get a point of comparison between the two implementations of Giraffe described in section 3.1.4. Both the architectures were implemented under the exact same conditions and on the exact data in order to get a fair comparison. The Segmented Giraffe model makes use of split input layers whereas the Combined Giraffe model is a fully connected network. This section discusses the observations and results of this experiment.

Starting off with the Segmented Giraffe network, clear signs of overfitting can be observed. A divergent trend exists between the training and test loss. A similar trend can be observed for the training and test accuracies. The network obtains a maximum test accuracy of 65.5%.

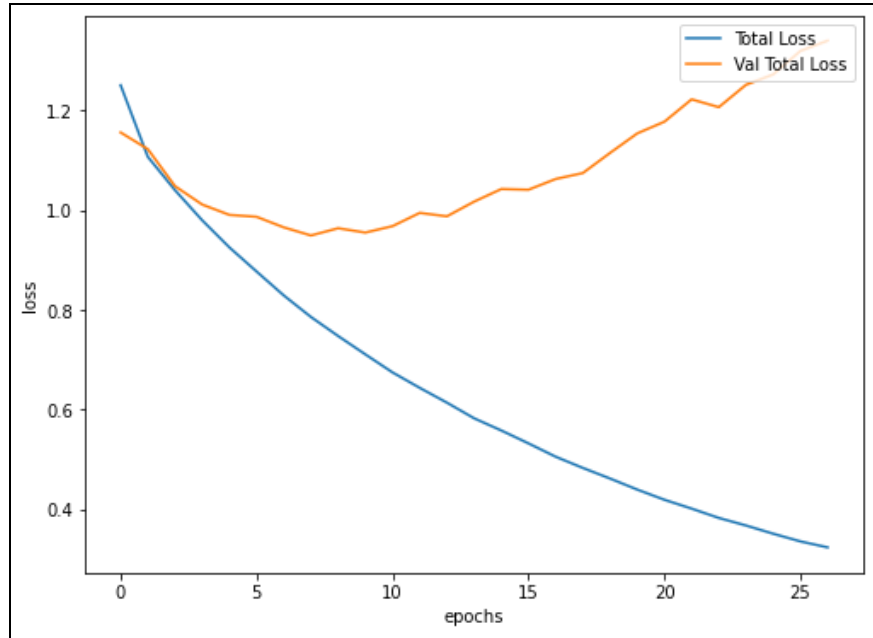


Figure 9. Loss vs Epochs curve for Segmented Giraffe

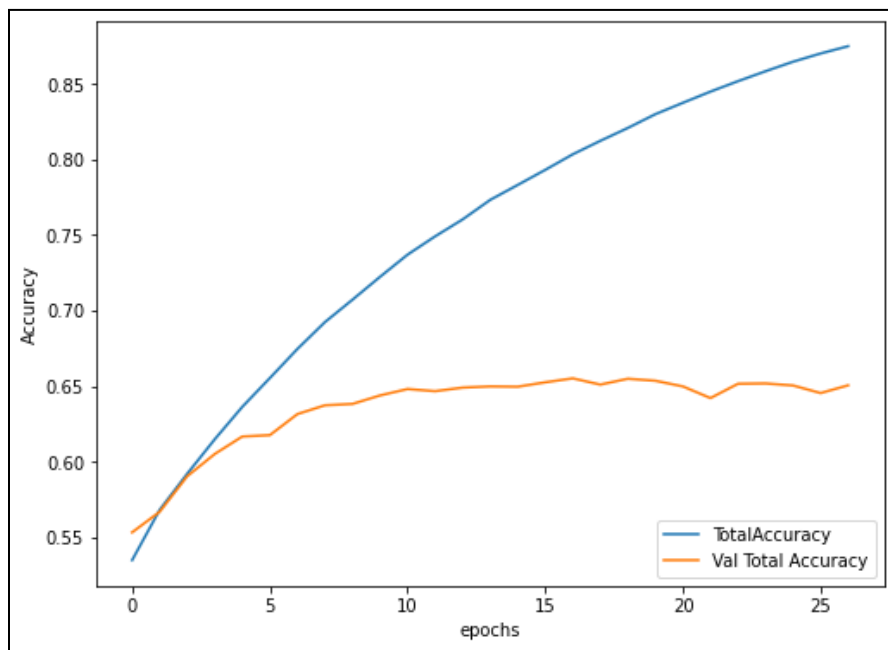


Figure 10. Accuracy vs Epochs curve for Segmented Giraffe

Moving on to the Combined Giraffe network, it can be seen that the network trains well but just like the segmented Giraffe is plagued by overfitting. However, this network marginally outperforms the previous network in terms of both testing accuracy and testing loss. The testing accuracy improves from 65.5 % to 66.2 % and the loss decreases from 0.949 to 0.927.

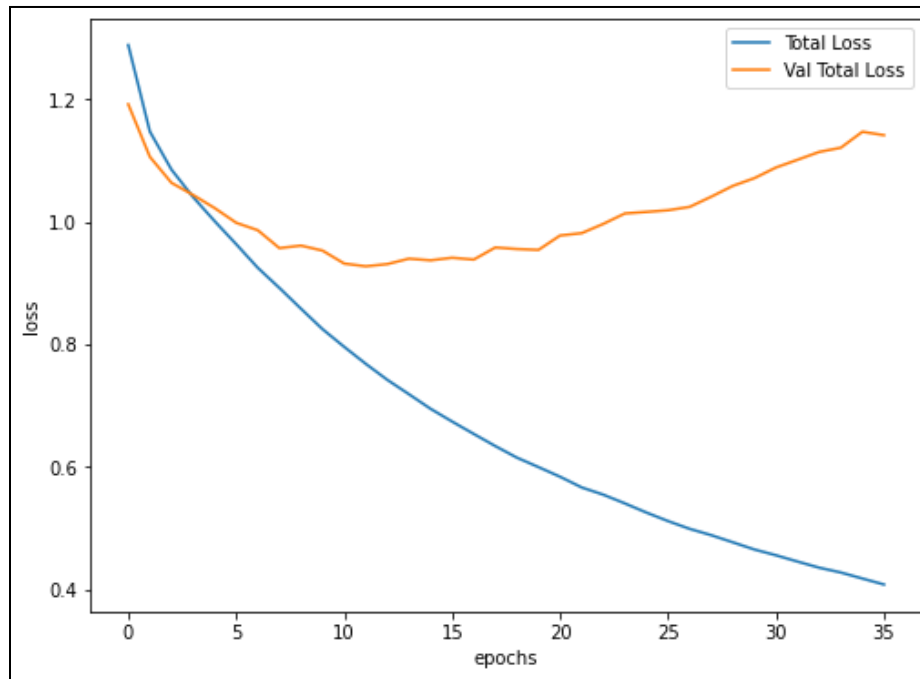


Figure 11. Loss vs Epochs curve for Combined Giraffe

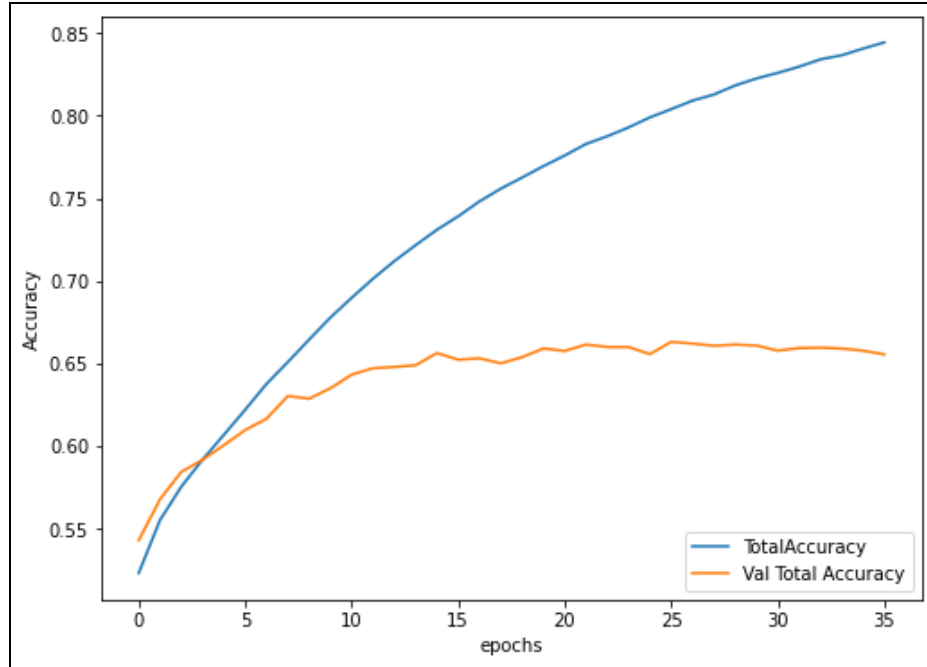


Figure 12. Accuracy vs Epochs curve for Combined Giraffe

Model	Metric	Value
Segmented Giraffe	Training Accuracy	0.874
	Testing Accuracy	0.655
	Training Loss	0.323
	Testing Loss	0.949
Combined Giraffe	Training Accuracy	0.844
	Testing Accuracy	0.662
	Training Loss	0.407
	Testing Loss	0.927

Table 7. Segmented and Combined Giraffe performance summary

4.1.2. Experiment 2: Effects of Regularization on Segmented and Combined Giraffe

This experiment was set up in order to compare the effects of regularization on both implementations from Experiment 1 and help prevent the imminent overfitting of in both architectures. Dropout layers were added to each architecture in order to perform this experiment. Both the architectures were implemented under the exact same conditions and on the exact data in order to get a fair comparison.

Starting off with the Segmented Regularized Giraffe, it can be observed that the network trains well without overfitting. Furthermore, this network outperforms its non-regularized implementation by obtaining a testing accuracy of 66.4% and a loss of 0.910. This goes on to show that regularization has a positive effect on the Segmented Giraffe implementation.

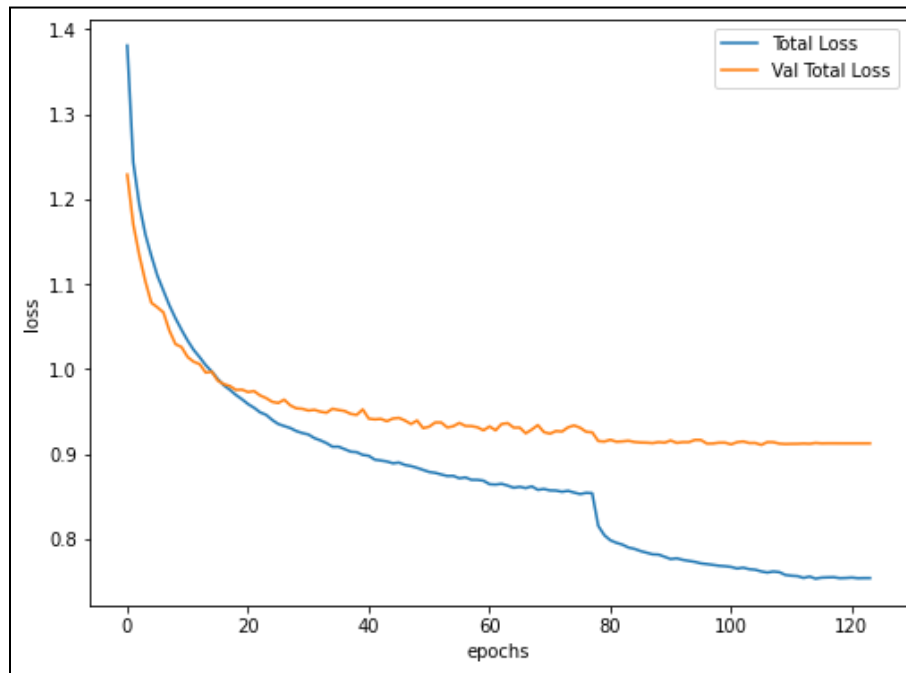


Figure 13. Loss vs Epochs curve for Segmented Regularized Giraffe

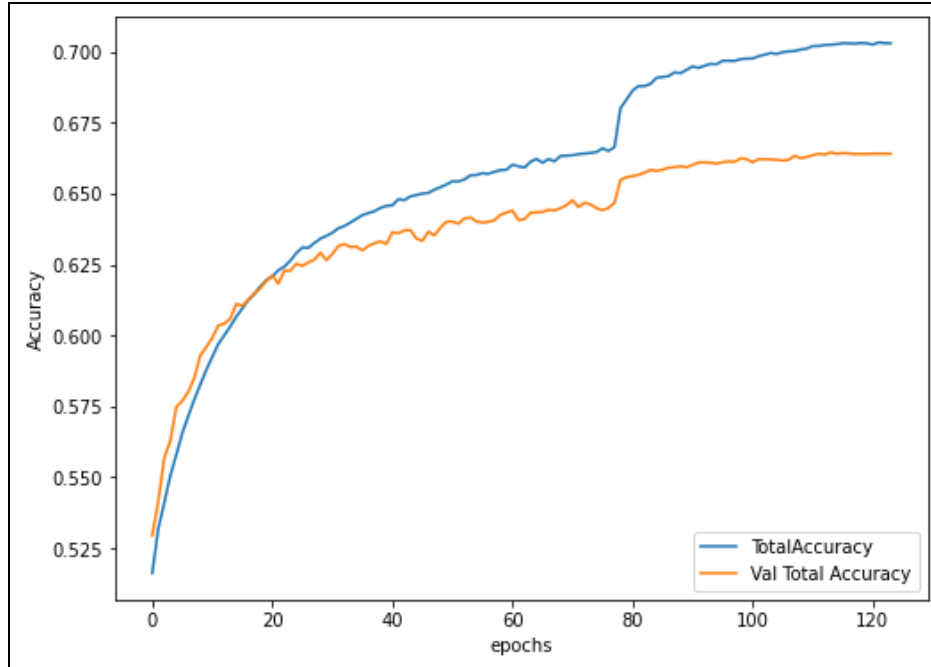


Figure 14. Accuracy vs epochs curve for Segmented Regularized Giraffe

However, the same cannot be said for the Combined Giraffe network. The network performance drops from 66.2% to 63.7%. Similar trends can be observed in all other metrics as well. This implies that the dropout out neurons had a significant contribution to performance for this architecture thus using either a small dropout ratio or other forms of regularization could be helpful.

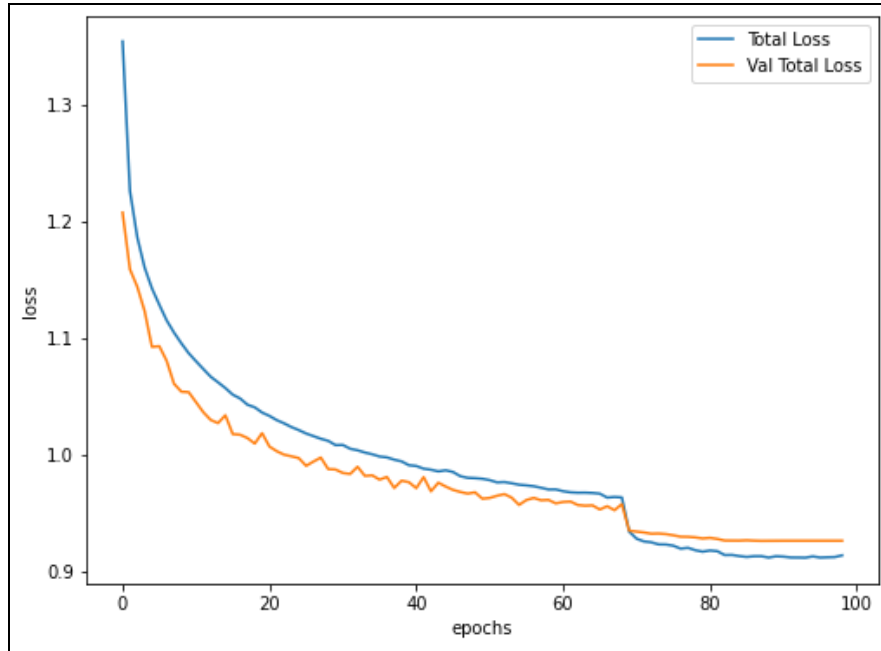


Figure 15. Loss vs Epochs curve for Combined Regularized Giraffe

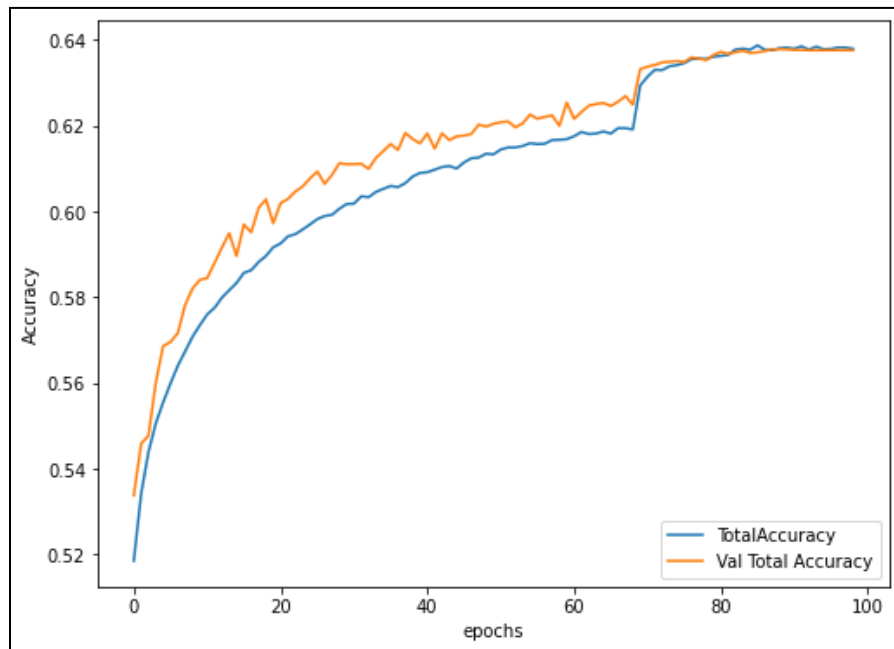


Figure 16. Accuracy vs Epochs curve for Combined Regularized Giraffe

Therefore, the Segmented Regularized Giraffe is the best architecture from among the four that were implemented with an accuracy of 66.4%. Architectures of both regularized networks are the same as their respective non-regularized versions but with dropout layers added.

Model	Metric	Value
Segmented Regularized Giraffe	Training Accuracy	0.703
	Testing Accuracy	0.664
	Training Loss	0.753
	Testing Loss	0.910
Combined Regularized Giraffe	Training Accuracy	0.638
	Testing Accuracy	0.637
	Training Loss	0.911
	Testing Loss	0.925

Table 8. Segmented Regularized and Combined Regularized Giraffe performance summary

4.2. DeepChess

This section provides a detailed description of the experiments conducted for the DeepChess network architecture.

DeepChess was implemented as a binary classification problem. The input comprised 100,000 randomly sampled positions with an equal win split between the two colours. Rectified linear units activation was implemented for all layers except the final layer which made use of the

softmax activation. A base learning rate of 0.005 was chosen and decreased by 1% every epoch. Two different pos2vec training methodologies were explored with two different sets of parameters resulting in a total of four implementations, each of which will be discussed in the experiments below. Moreover, another experiment was set up wherein this architecture was implemented on a much bigger but lower quality database to establish a point of comparison with respect to the original database. Due to computational limitations, each of these networks was only trained for 50 epochs each.

4.2.1. Experiment 1: Effect of Optimizer and Loss Function in DBN training (pos2vec)

This experiment was set up in order to compare two sets of training parameters. The first set implements the adam optimizer in conjunction with the binary cross-entropy loss. Whereas the second set implements the stochastic gradient descent (SGD) optimizer along with the mean squared error loss function. Both cases were implemented under the exact same conditions and on the exact same data in order to get a fair comparison. The model architecture implemented for both cases is a simple DBN training without decoder layer freezing.

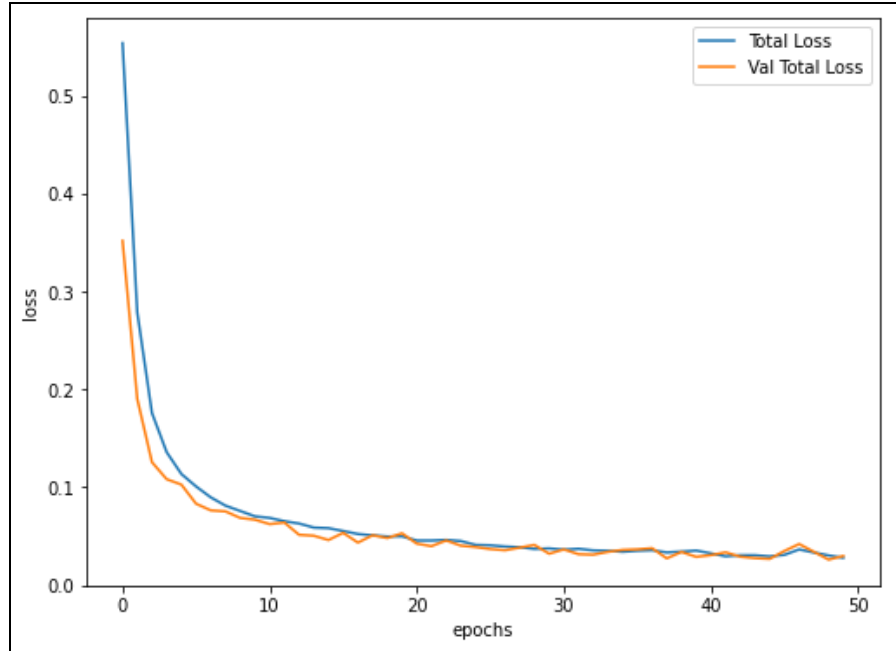


Figure 17. Loss vs Epochs curve for DeepChess with DBN trained using adam optimizer and binary cross-entropy loss

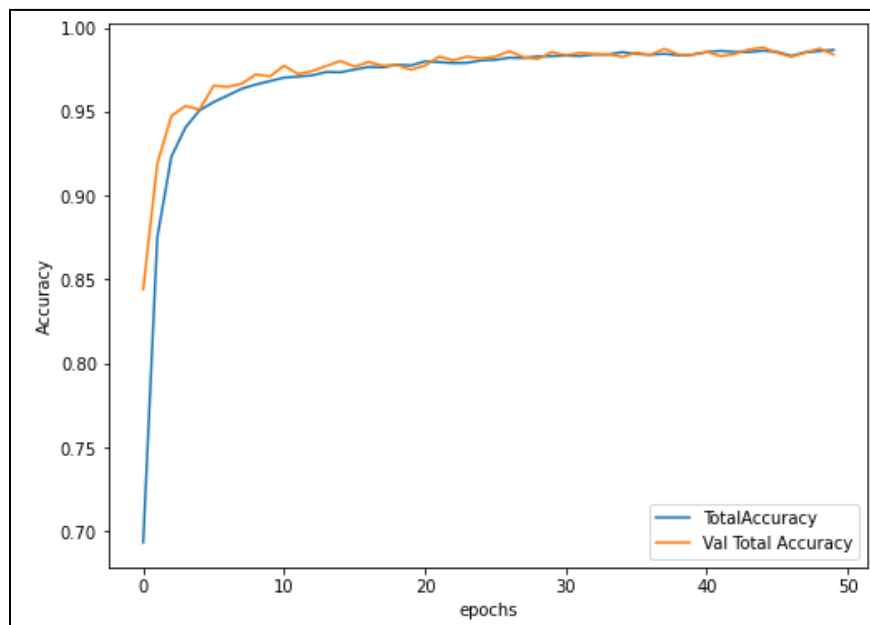


Figure 18. Accuracy vs Epochs curve for DeepChess with DBN trained using adam optimizer and binary cross-entropy loss

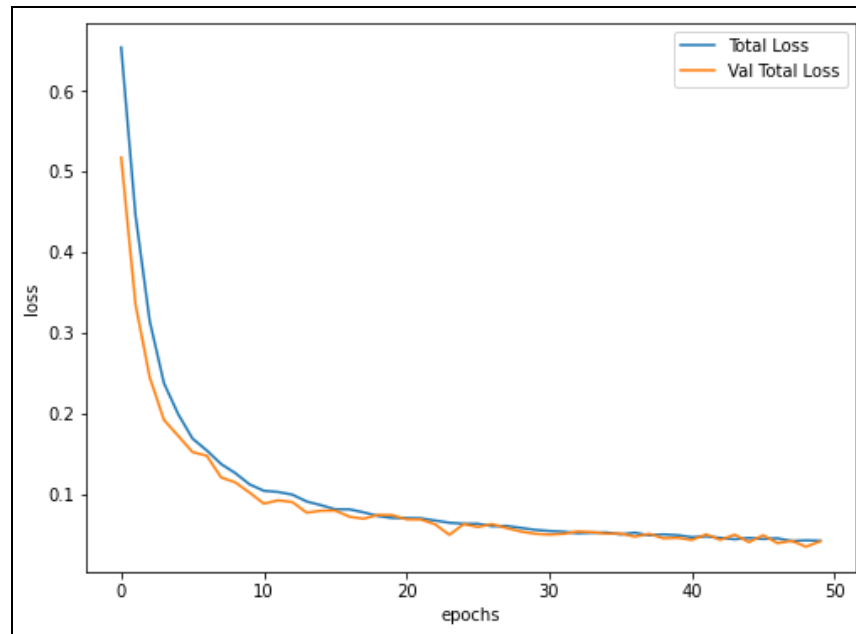


Figure 19. Loss vs Epochs curve for DeepChess with DBN trained using SGD optimizer and MSE loss

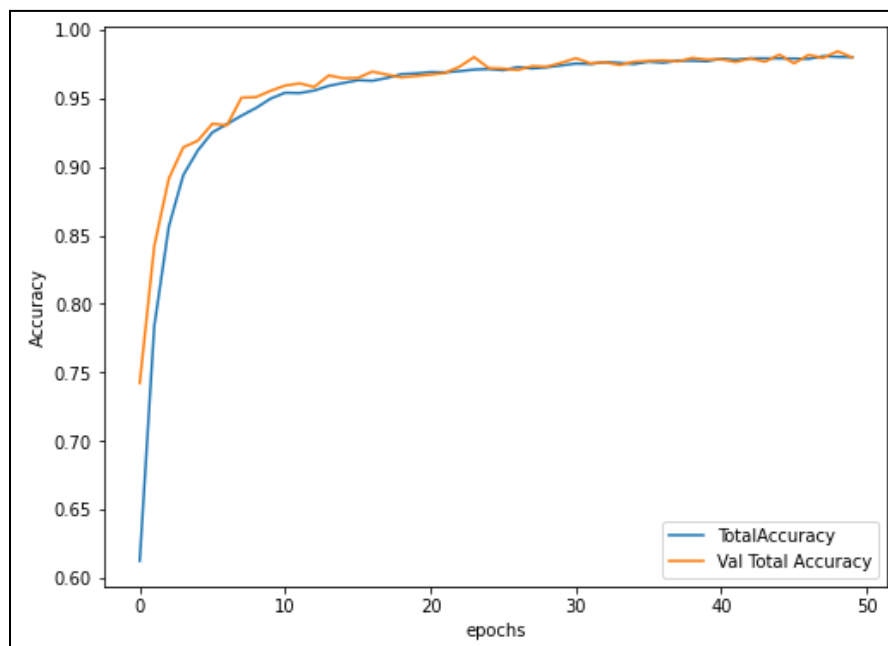


Figure 20. Accuracy vs Epochs curve for DeepChess with DBN trained using SGD optimizer and MSE loss

Model	Metric	Value
DeepChess with DBN trained using Adam and Binary Cross Entropy	Training Accuracy	0.986
	Testing Accuracy	0.988
	Training Loss	0.027
	Testing Loss	0.026
DeepChess with DBN trained using SGD and MSE loss	Training Accuracy	0.980
	Testing Accuracy	0.984
	Training Loss	0.042
	Testing Loss	0.034

Table 9. DeepChess with DBN trained using different sets of parameters performance summary

Having observed the results of both these networks it is quite discernable that the DeepChess model trained with adam and binary cross-entropy outperforms the other network across all metrics. It obtains a testing accuracy of 98.8% with a testing loss of 0.026 whereas the other network achieves a testing accuracy of 98.4% with a testing loss of 0.034. Thus moving forward, all models will make use of these parameters.

4.2.2. Experiment 2: Effects of Freezing Decoder Layers in DBN training (pos2vec)

This experiment was set up in order to compare and analyze the two ways in which the DeepChess DBN can be trained. The first implementation involves freezing the weights of the decoder of the previous layer so as to preserve the information it contains for future training rounds. Whereas the second implementation involves not freezing the layers and letting them train together. This is also the implementation that was used for Experiment 1 in section 4.2.1. Both cases were implemented under the exact same conditions and on the exact same data in order to get a fair comparison.

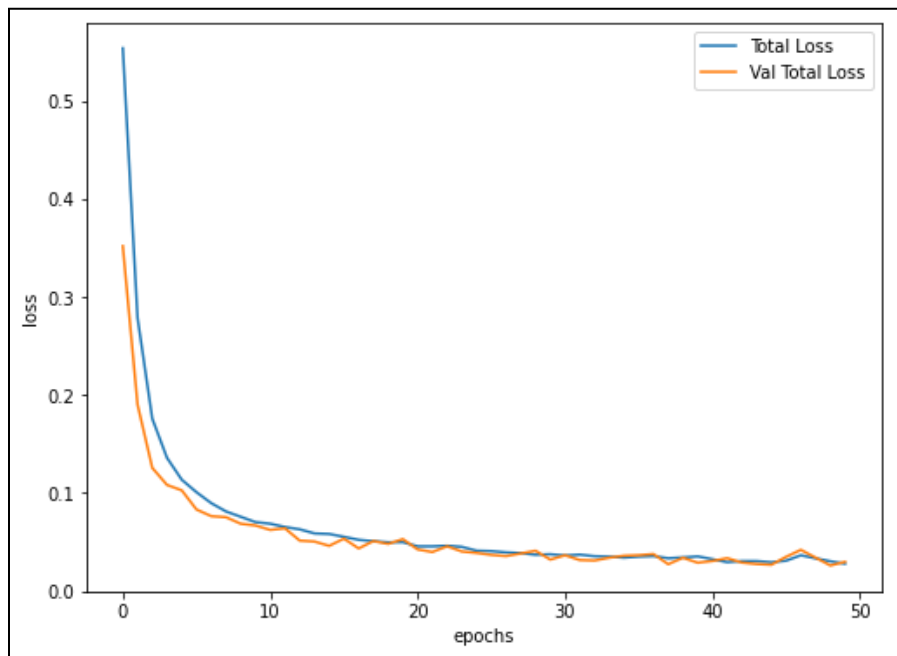


Figure 21. Loss vs Epochs curve for DeepChess with DBN trained without freezing decoder layers

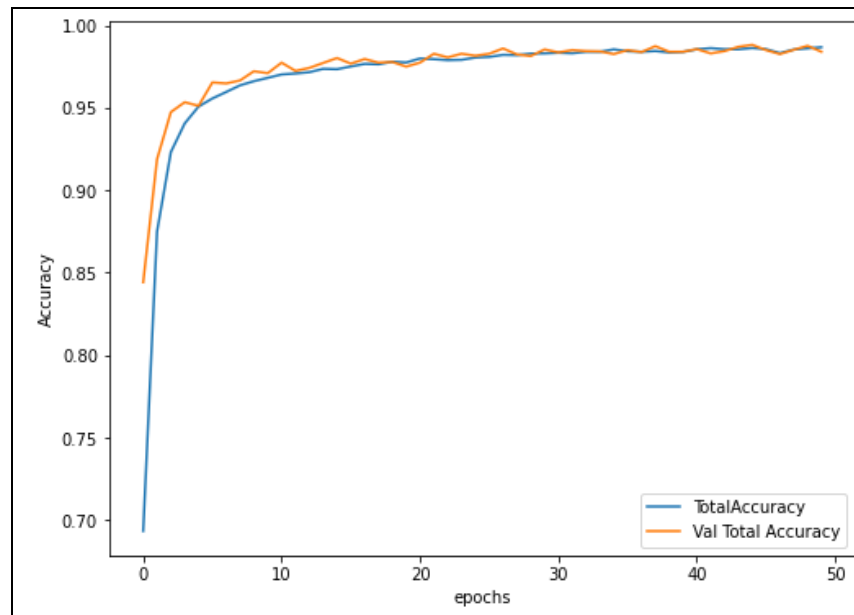


Figure 22. Accuracy vs Epochs curve for DeepChess with DBN trained without freezing decoder layers

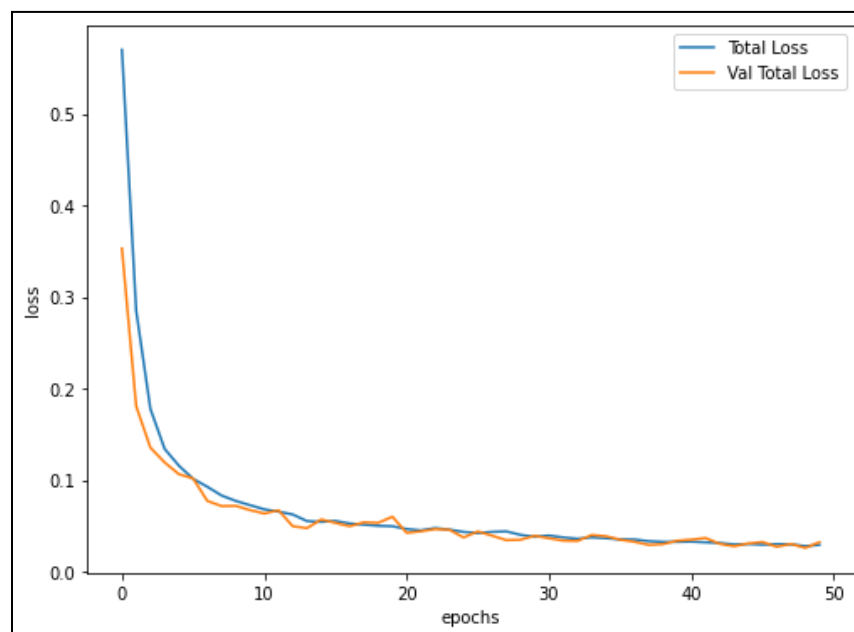


Figure 23. Loss vs Epochs curve for DeepChess with DBN trained with freezing decoder layers

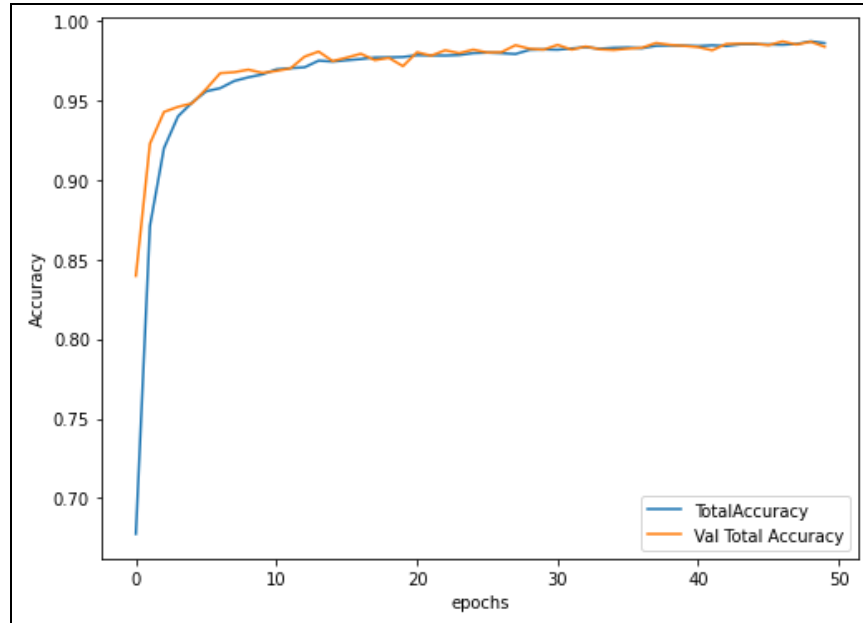


Figure 24. Accuracy vs Epochs curve for DeepChess with DBN trained with freezing decoder layers

Model	Metric	Value
DeepChess with DBN trained without freezing decoder layers	Training Accuracy	0.986
	Testing Accuracy	0.988
	Training Loss	0.027
	Testing Loss	0.026
DeepChess with DBN trained with freezing decoder layers	Training Accuracy	0.987
	Testing Accuracy	0.987
	Training Loss	0.027
	Testing Loss	0.026

Table 10. DeepChess with DBN trained with and without freezing decoder layers performance summary

Having observed the results of both these networks, it is fair to say that the results are extremely similar to each other. This could be attributed to the point that freezing the decoder layers during pos2vec training does not affect the DeepChess results to a discernible degree. This implies that even after freezing certain layers in the DBN training process the overall DeepChess model was able to maintain its performance. Therefore, in such a situation, the network with the frozen layers is preferable as it takes relatively less time to train and is able to optimize the performance while only employing a subset of the feature space. The same experiment was also performed on the SGD and MSE architecture from Experiment 1 in section 4.2.1 and similar results were obtained.

4.2.3. Experiment 3: Effects of a Bigger Dataset

Deepchess is a deep learning network and thus usually requires thousands if not millions of samples to train a dependable and generalized model. In all the previous iterations of DeepChess mentioned in this report, it has been implemented using 50,000 pairs of training samples and 5,000 pairs of testing samples. These are relatively small data sets to train a deep learning network. The reason for this dataset being quite small is that this dataset is from grandmaster's games at the highest level and almost half of the games at this level end up in draws which are not included in the DeepChess data.

Even though we are able to obtain great performance, these networks' ability to generalize may be compromised due to a smaller test set. Thus, in order to create a more dependable and generalizable implementation of DeepChess, a data set of over 100,000 games was created. These games were played on the platform Lichess by players at rating 2200+. They were obtained from the Elite Lichess Database and were played in January of 2021. This dataset was

almost 17 times bigger than the previous one. All of the other data preparation and model execution steps were exactly the same as the previously described implementations.

The new data used consisted of 1,100,000 positions, 550,000 of which were white wins and the other half were black wins. This data was split up into training and testing sets. Training data was made up of 1,000,000 positions with 500,000 wins for each colour. Similarly, the testing data had 100,000 positions with 50,000 wins for each colour. Therefore, the input to the DeepChess model was 500,000 pairs of positions. Due to computational limitations, this network was trained for 500 epochs. The architecture of choice was DeepChess with DBN trained with freezing decoder layers.

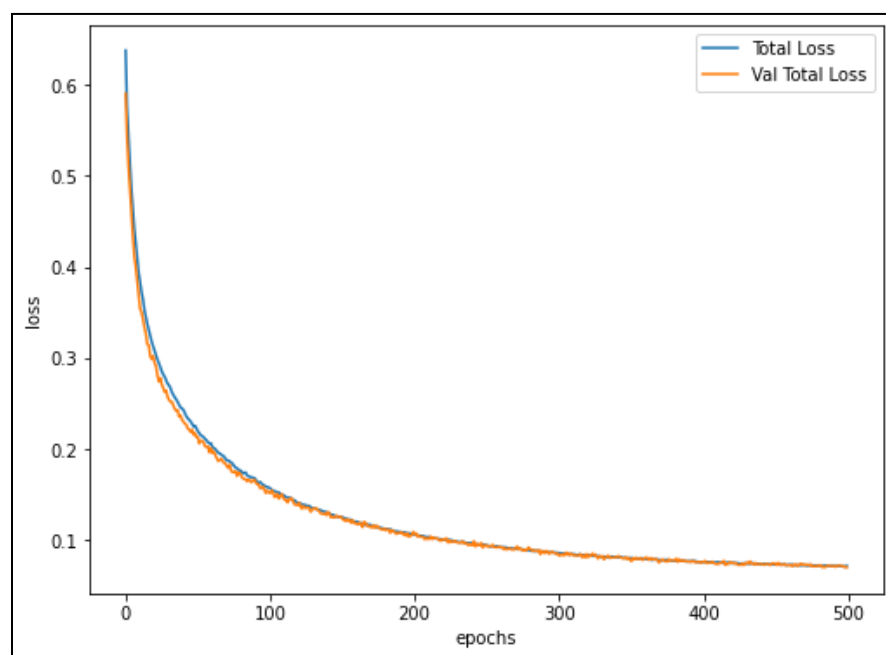


Figure 25. Loss vs Epochs curve for DeepChess trained on LiChess Dataset

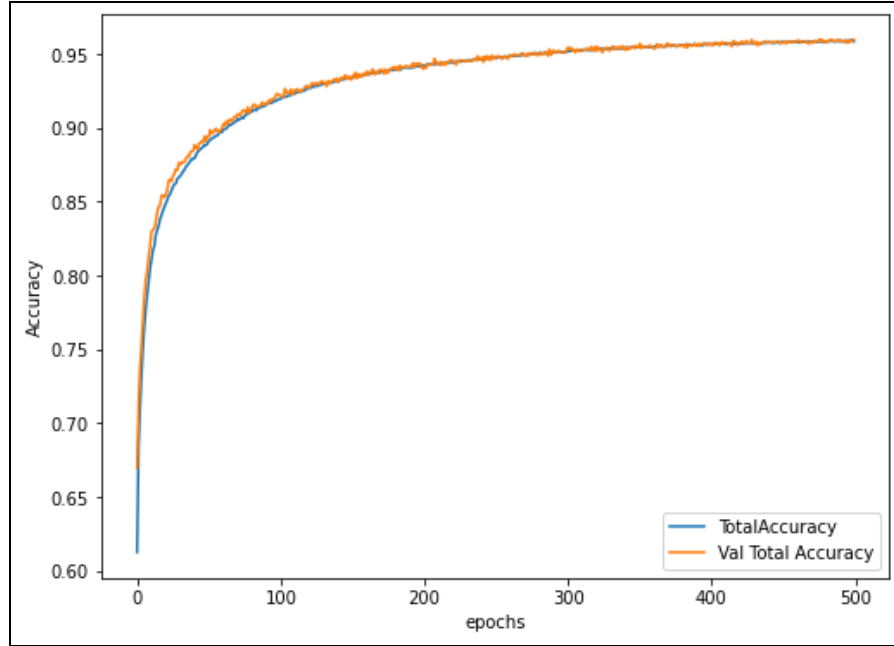


Figure 26. Accuracy vs Epochs curve for DeepChess trained on LiChess Dataset

Model	Metric	Value
LiChess - DeepChess with DBN trained with freezing decoder layers	Training Accuracy	0.959
	Testing Accuracy	0.960
	Training Loss	0.070
	Testing Loss	0.069

Table 11. DeepChess trained on LiChess dataset performance summary

This network achieves a testing accuracy of 96% and a testing loss of 0.69. This is a much more realistic estimation of DeepChess architecture’s true performance as this network has been tested over a much larger and diverse test set as compared to the previous implementations of DeepChess.

5. CONCLUSION

This report implements two end to end deep learning pipelines for the evaluation of chess. Deep neural networks are primarily used to replicate human intuition in computer chess. Giraffe achieves an accuracy of 66.4% with the segmented regularized architecture. This method implements a unique feature extraction which allows it to aid the network in learning certain concepts like promotions, tempo, piece cohesion, etc [1]. This implementation is a great example of combining conventional and modern chess engines as it makes use of deep learning while still making use of a rule-based feature selection. Overall, this model can with relative certainty classify the evaluation of a chessboard at any given position. An end to end implementation of DeepChess is also detailed in this report. It achieves an accuracy of 98.8% on the smaller dataset and an accuracy of 96% on the larger LiChess dataset. This method stands out as a fully modern evaluation function because it evaluates chess positions using a deep neural network without any a priori knowledge regarding the rules of chess [2]. All in all, this model can with almost absolute certainty identify the best position for a win between any two given chess positions. To summarize, two deep learning pipelines were implemented to create chess evaluation functions that can learn the rules and intricacies of the game by themselves from the database and can evaluate, and analyze chess more efficiently by comprehending complex strategic ideas that are intuitive to humans but have been difficult for computers to grasp for a long time.

An interesting topic for future research would be to combine the feature representation from Giraffe into the network architecture from DeepChess. This is primarily useful because as per Giraffe's author, its feature representation is superior to bitboard based representation used for the DeepChess model [1]. This could lead to an improvement in how DeepChess deals with certain chess concepts that aren't particularly easy to grasp through bitboards but are proven to

be reflected in the Giraffe representation like promotions, tempo, piece cohesion, etc. Furthermore, both networks can be improved upon by incorporating detailed and thorough hyperparameter tuning. Although some tuning was incorporated in the current implementations, due to lack of time not all parameters were tuned to their best possible values. Moreover, for the DeepChess implementation, the performance could be marginally improved by training the networks for more epochs. The training was restricted in the above-mentioned implementations due to computational limitations.

References

- [1] M. Lai, "Giraffe: Using deep reinforcement learning to play chess," *arXiv preprint arXiv:1509.01549*, 2015.
- [2] O. E. David, N. S. Netanyahu, and L. Wolf, "Deepchess: End-to-end deep neural network for automatic learning in chess," in *International Conference on Artificial Neural Networks*, 2016: Springer, pp. 88-96.
- [3] Basic AI Methods and Tools
- [4] J. v. Neumann, "Zur theorie der gesellschaftsspiele," *Mathematische annalen*, vol. 100, no. 1, pp. 295-320, 1928.
- [5] C. E. Shannon, "XXII. Programming a computer for playing chess," *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, vol. 41, no. 314, pp. 256-275, 1950.
- [6] F.-H. Hsu, *Behind Deep Blue: Building the computer that defeated the world chess champion*. Princeton University Press, 2002.
- [7] Romstad, Costalba, Kiiski, Yang, Spitaleri and Ablett, "Stockfish, open source chess engine". 2011

Appendix

Giraffe Segmented Implementation:

```
def giraffe_segmented():
    input1 = Input(shape=(20,))
    input2 = Input(shape=(160,))
    input3 = Input(shape=(176,))
    x1= Dense(16, activation = 'relu')(input1)
    x1= Dense(8, activation = 'relu')(x1)
    x2= Dense(128, activation = 'relu')(input2)
    x2= Dense(64, activation = 'relu')(x2)
    x3= Dense(512, activation = 'relu')(input3)
    x3= Dense(256, activation = 'relu')(x3)
    merged = Concatenate(axis=1)([x1,x2,x3])
    l1 = Dense(1024, activation='relu')(merged)
    l2 = Dense(512, activation='relu')(l1)
    l3 = Dense(256, activation='relu')(l2)
    l4 = Dense(128, activation='relu')(l3)
    l5 = Dense(64, activation='relu')(l4)
    l6 = Dense(32, activation='relu')(l5)
    l7 = Dense(14, activation='relu')(l6)
    out = Dense(7, activation='softmax')(l7)
    model_giraffe = Model(inputs=[input1, input2, input3], outputs=out)
```

Giraffe Combined Implementation:

```
def giraffe_combined():
    model = Sequential()
    model.add(Dense(1024, activation='relu', input_shape = (356,)))
    model.add(Dense(512,activation='relu'))
    model.add(Dense(256,activation='relu'))
    model.add(Dense(128,activation='relu'))
    model.add(Dense(64,activation='relu'))
    model.add(Dense(32,activation='relu'))
    model.add(Dense(14,activation='relu'))
    model.add(Dense(7,activation='softmax'))
```

Giraffe Segmented Regularized Implementation:

```
def giraffe_segmented_regularized():
    input1 = Input(shape=(20,))
    input2 = Input(shape=(160,))
    input3 = Input(shape=(176,))
    x1= Dense(16, activation = 'relu')(input1)
    r1= Dropout(0.2)(x1)
    x1= Dense(8, activation = 'relu')(r1)
    x2= Dense(128, activation = 'relu')(input2)
    r2= Dropout(0.2)(x2)
    x2= Dense(64, activation = 'relu')(r2)
    x3= Dense(512, activation = 'relu')(input3)
    r3= Dropout(0.2)(x3)
    x3= Dense(256, activation = 'relu')(r3)
    merged = Concatenate(axis=1)([x1,x2,x3])
    l1 = Dense(1024, activation='relu')(merged)
    d1 = Dropout(0.2)(l1)
    l2 = Dense(512, activation='relu')(d1)
    d2= Dropout(0.2)(l2)
    l3 = Dense(256, activation='relu')(d2)
    d3= Dropout(0.2)(l3)
    l4 = Dense(128, activation='relu')(d3)
    d4= Dropout(0.2)(l4)
    l5 = Dense(64, activation='relu')(d4)
    d5= Dropout(0.2)(l5)
    l6 = Dense(32, activation='relu')(d5)
    d6= Dropout(0.2)(l6)
    l7 = Dense(14, activation='relu')(d6)
    d7= Dropout(0.2)(l7)
    out = Dense(7, activation='softmax')(d7)
    model_giraffe = Model(inputs=[input1, input2, input3], outputs=out)
```

Giraffe Combined Regularized Implementation:

```
def giraffe_combined_regularized():
    model = Sequential()
    model.add(Dense(1024, activation='relu', input_shape = (356,)))
    model.add(Dropout(0.2))
    model.add(Dense(512, activation='relu'))
    model.add(Dropout(0.2))
    model.add(Dense(256, activation='relu'))
    model.add(Dropout(0.2))
    model.add(Dense(128, activation='relu'))
    model.add(Dropout(0.2))
    model.add(Dense(64, activation='relu'))
    model.add(Dropout(0.2))
    model.add(Dense(32, activation='relu'))
    model.add(Dropout(0.2))
    model.add(Dense(14, activation='relu'))
    model.add(Dropout(0.2))
    model.add(Dense(7, activation='softmax'))
```

DeepChess DBN Implementation:

```
model = Sequential([
    Dense(773, activation='relu'),
    Dense(600, activation='relu'),
    Dense(400, activation='relu'),
    Dense(200, activation='relu'),
    Dense(100, activation='relu')
])
model.load_weights('C:/Users/manav/OneDrive/Desktop/FYP/Deepchess-11.01/models/fpos2vec/fpos2vec')
```

DeepChess Disjoint Siamese Network Implementation:

```
left_input = Input((None, 773))
right_input = Input((None, 773))
encoded_l = model(left_input)
encoded_r = model(right_input)
concatenated = Concatenate()([encoded_l, encoded_r])
deepchess = Dense(400, activation='relu')(concatenated)
deepchess = Dense(200, activation='relu')(deepchess)
deepchess = Dense(100, activation='relu')(deepchess)
deepchess_output = Dense(2, activation='softmax')(deepchess)

deepchess_model = Model(inputs=[left_input, right_input], outputs=deepchess_output)
```